

SeedHammer II — loading a systemwide payload

Putting records into the machine's **systemwide region** and using them: the host container, the delivery to flash, the boot offer, the digest comparison across the air gap, and the two ways back out. Host CLI on one side, the firmware GUI in the emulator on the other.

Test material only — never put funds behind these records.

The mnemonic is BIP-39's own all-zero-entropy vector (abandon...about) and the passphrase is the xkcd string. Both are published; the payload is UNSEALED, so there is no secret here to leak.

What a systemwide payload is

A container written once to a fixed 64 KiB flash region at 0x10D00000, holding records the machine's programs can draw on instead of asking the operator to type them. It is **not** the Sealed Payload — that is a different container (MNEMBLOB) in a different region (0xE1000000), and the two surfaces are kept apart so no invocation can produce one while the operator believes they are producing the other.

The firmware never writes flash.

Spec §13 D10. Every write in this document is the HOST talking to the RP2350 bootrom over USB with the machine in BOOTSEL. The device can *load* a payload into RAM and *unload* it again; it cannot put one there, and it cannot remove one. That asymmetry is the reason the last screen in this document says "Still in flash. Wipe it from the host."

The three records

record	class	what it is for
abandon abandon ... about	ClassMnemonic	a seed the Backup Wallet and Engrave Seed programs can take directly
pass:636f7272...	ClassPassphrase	a BIP-39 passphrase, offered where one is asked for
text:53454544...	ClassFreeText	free text, offered to Engrave Text

Three classes rather than one on purpose: a single-record payload loads, shows a digest, and demonstrates nothing. The property worth walking is that **one payload feeds several programs**, each picker defaulting to FROM PAYLOAD for the class it wants.

Toolchain

```
$ target/release/me --version
me 0.7.0
[exit 0]
```

Host step 1 — the records file

inputs-payload/records.txt

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
pass:636f727265637420686f727365206261747465727920737461706c65
text:5345454448414d4d45522049492044454d4f205041594c4f4144
```

Obstacle — the obvious way to write it is refused

Nobody writes hex the first time. They write the passphrase and the free text as themselves, and the container is refused with **exit 4**:

inputs-payload/records-as-first-written.txt

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
pass:correct horse battery staple
text:SEEDHAMMER II DEMO PAYLOAD
```

```
$ target/release/me sysw pack --no-passphrase --in inputs-payload/records-as-first-written.txt --out out/rejected.bin
strength: no passphrase — BELOW the threshold
me: WARNING — this payload carries secret material with weak or no passphrase protection. Proceeding (spec §13 D3).
me: record 1 (records count from 0) begins `pass:`, but its body is not lowercase hex. That prefix is RESERVED, so a body it cannot decode is refused rather than quietly engraved as free text (§5.3.1). Encode the body first:
    printf '%s' 'your text here' | xxd -p -c 256
[exit 4]
```

Why the bodies are hex at all — the least obvious rule here.

EPD §6.4 requires every record to be the canonical unbroken string — no interior spaces, no hyphens, no grouping — and uses LF as the record separator on the grounds that no constellation string contains a newline. `correct horse battery staple` has three spaces and Engrave Text's keyboard has a newline key, so both new classes break both clauses. Records are engraved VERBATIM, so relaxing §6.4 would turn a scratch on the operator's only copy into silently-absorbed damage. Lowercase rather than any hex, because EPD §6.6 hashes the section lowercased and hex is the only common encoding that survives lowercasing unchanged.

This message is better than it was, and this run is why.

It used to read *"record 1 is not a form this container can place. Descriptors and addresses are not yet classifiable here"* — true of a descriptor, and not true of what had actually happened. `classify` returns Unknown for two unrelated situations and the message named only the one that did not apply, sending the reader to look for a parser problem instead of typing one `xxd`. Fixed in `me` (1538ef0); the error now names the prefix and the remedy, and deliberately still does not print the body — a `pass:` body is a passphrase, and `stderr` is scrolled back, logged and pasted into bug reports.

Host step 2 — so the bodies are encoded

```
$ bash -c printf 'correct horse battery staple' | xxd -p -c 256
636f727265637420686f727365206261747465727920737461706c65
[exit 0]
```

```
$ bash -c printf 'SEEDHAMMER II DEMO PAYLOAD' | xxd -p -c 256
5345454448414d4d45522049492044454d4f205041594c4f4144
[exit 0]
```

Host step 3 — pack, and the digest that crosses the air gap

```
$ target/release/me sysw pack --no-passphrase --in inputs-payload/records.txt --out out/payload.bin
strength: no passphrase — BELOW the threshold
me: WARNING — this payload carries secret material with weak or no passphrase protection. Proceeding (spec §13 D3).
digest: 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5
[exit 0]

$ stat -c %n %s bytes out/payload.bin
out/payload.bin 265 bytes
[exit 0]

$ bash -c xxd -l 16 'out/payload.bin'
00000000: 4d4e 454d 5359 5357 0100 0000 0000 0000  MNEMSYSW.....
... clipped; full text in out/transcript_payload.txt
```

```
$ target/release/me sysw show out/payload.bin
sealed: false
pub_len: 213
ct_len: 0
identity: deb91a0dec3d86d642e9f2cee0ea5b02c80088e814f4fb20ffb98d54eb321318
digest: 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5
[exit 0]
```

The warning is correct and worth reading rather than dismissing.

This payload carries a mnemonic and a passphrase with `--no-passphrase`, so they sit in flash in the clear. `me` warns and proceeds (§13 D3) instead of refusing: for a published test vector that is the right call, and for a real seed the operator has just been told exactly what they are about to do.

The same bytes the emulator embeds

The device screenshots later in this document are of *this* container, and that is checked rather than asserted — the journey would otherwise be wrong in the one way a reader cannot detect, since it would still read as consistent:

```
$ cmp out/payload.bin seedhammer/cmd/emu/sysw_test_payload.bin
[exit 0]

$ bash -c sha256sum 'out/payload.bin' 'seedhammer/cmd/emu/sysw_test_payload.bin'
b575baad2c4d9fd8d36e34f9341ca77d6242bad1dfbc9bdceffa93a64277882 out/payload.bin
b575baad2c4d9fd8d36e34f9341ca77d6242bad1dfbc9bdceffa93a64277882 seedhammer/cmd/emu/sysw_test_payload.bin
[exit 0]

$ grep syswTestDigest seedhammer/cmd/emu/sysw_test_payload.go
64:const syswTestDigest = "55ad b800 6ec6 a066 94f3 6a0e 900a c8d5"
[exit 0]
```

Host step 4 — the region image, and what gets written

```
$ target/release/me sysw pack --no-passphrase --in inputs-payload/records.txt --region --out out/payload-region.bin
strength: no passphrase – BELOW the threshold
me: WARNING – this payload carries secret material with weak or no passphrase protection. Proceeding (spec §13 D3).
digest: 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5
me: region image – 265 bytes of container, padded with 0xFF to 65536; write it at 0x10D00000
[exit 0]

$ stat -c %n %s bytes out/payload-region.bin
out/payload-region.bin 65536 bytes
[exit 0]

$ bash -c xxd -s 260 -l 32 'out/payload-region.bin'
00000104: 3036 6336 35ff ffff ffff ffff ffff ffff 06c65.....
00000114: ffff ffff ffff ffff ffff ffff ffff ffff .....
[exit 0]
```

The tail is 0xFF, not zeros.

That is the ERASED state of NOR flash, so the image is byte-for-byte what the sector looks like with only the container in it. Zero-padding would be a WRITE of 64 KiB to say nothing.

Padding does not change the digest

The digest is over the container's public records, not over the region image, so the value the operator compares on the device is the same whichever form was written. Both show calls above and below print 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5.

```
$ target/release/me sysw show out/payload-region.bin
sealed: false
pub_len: 213
ct_len: 0
identity: deb91a0dec3d86d642e9f2cee0ea5b02c80088e814f4fb20ffb98d54eb321318
digest: 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5
[exit 0]
```

Host step 5 — delivery, with the machine in BOOTSEL

```
$ bash -c cd 'seedhammer' && /nix/var/nix/profiles/default/bin/nix develop --command picotool version
picotool v2.2.0-a4 (Linux, GNU-15.2.0, Release)
[exit 0]

$ picotool load --verify -t bin -o 0x10D00000 out/payload-region.bin
(not run here: it requires the machine attached in BOOTSEL. It HAS been
run -- the payload is on the engraver, and the device shows the digest
section 5 printed above.)
```

Removing one — also the host, and also not "erase"

The device offers UNLOAD, which drops the payload from RAM. The region is untouched by that, and the firmware has no code path that could touch it. To overwrite the region you write another image over it, exactly the way the first one arrived:

```
$ target/release/me sysw wipe --fill ones --out out/wipe-ones.bin
me: note — 0xFF is the ERASED state of NOR flash, so this region will be indistinguishable from one that was never written
[exit 0]

$ target/release/me sysw wipe --out out/wipe-random.bin
[exit 0]

$ stat -c %n %s bytes out/wipe-ones.bin out/wipe-random.bin
out/wipe-ones.bin 65536 bytes
out/wipe-random.bin 65536 bytes
[exit 0]

$ bash -c xxd -l 16 'out/wipe-ones.bin'
00000000: ffff ffff ffff ffff ffff ffff ffff ffff .....
[exit 0]

$ bash -c xxd -l 16 'out/wipe-random.bin'
00000000: f313 b59c 5456 c3e6 fe7a 9ccf 9d3b 1238 ....TV...z...;8
[exit 0]

$ target/release/me sysw show out/wipe-ones.bin
me: not a systemwide container: BadMagic
[exit 4]
```

--fill ones is not the safe default, and it is not the paranoid one either — it is a choice about what a later reader can conclude.

0xFF leaves the region indistinguishable from one that was never written, which is what you want if the existence of a payload is itself sensitive. Random fill leaves it visibly overwritten. me defaults to random and says the other one's consequence out loud.

The diagram illustrates a three-step process for loading a systemwide payload:

- Step 1: Payload**
A green screen with the title "Payload". In the center, there are two buttons: "LOAD" (highlighted in white) and "SKIP" (in green). At the bottom, it says "A systemwide payload is present. Load it?". A blue checkmark icon is in the bottom right corner. A white arrow points right from the left edge.
- Step 2: Payload**
A green screen with the title "Payload". In the center, there are two buttons: "LOAD" (in green) and "SKIP" (highlighted in white). At the bottom, it says "A systemwide payload is present. Load it?". A blue checkmark icon is in the bottom right corner. A white arrow points right from the left edge.
- Step 3: Payload Digest**
A green screen with the title "Payload Digest". It says "Compare this against what 'me sysw pack' printed:" followed by a long alphanumeric string: "55ad b800 6ec6 a066 94f3 6a0e 900a c8d5". A blue checkmark icon is in the bottom right corner. A white arrow points right from the left edge.

```
HOST $ me sysw show out/payload.bin
digest: 55ad b800 6ec6 a066 94f3 6a0e 900a c8d5

FORK cmd/emu/sysw_test_payload.go:64
const syswTestDigest = "55ad b800 6ec6 a066 94f3 6a0e 900
a c8d5"

DEVICE the screen to the right
```

The device screen shows a green background with the text "Payload Digest" in white. Below it, it says "Compare this against what" and "me sysw pack` printed:". The calculated digest "55ad b800 6ec6 a066 94f3 6a0e 900a c8d5" is displayed in white. A blue checkmark icon is visible in the bottom right corner of the screen.

Backing out of the digest screen does not leave the payload half-loaded and does not silently continue. It says which of those it did.

The diagram illustrates the three screens of the payload loading process, each represented by a green rectangular screen with a blue checkmark button in the bottom right corner.

- Not Loaded:** The screen displays the text "Not Loaded" at the top. Below it, the message "Digest not compared. Nothing was loaded." is shown. A blue button with a white checkmark is in the bottom right corner.
- Payload Warnings:** The screen displays the text "Payload Warnings" at the top. Below it, the message "A SECRET is stored unencrypted in flash." is shown. A blue button with a white checkmark is in the bottom right corner.
- Payload:** The screen displays the text "Payload" at the top. Below it, the message "Keep this payload loaded?" is shown. A blue button with a white checkmark is in the bottom right corner.

Arrows indicate the flow from "Not Loaded" to "Payload Warnings" and from "Payload Warnings" to "Payload".

Not Loaded — "Digest not compared. Nothing was loaded." Accepting → the warnings, stated before the payload is live KEEP / UNLOAD — the last chance before it is loaded

On the machine — what a loaded payload changes

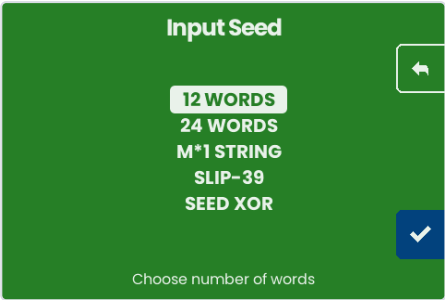
This is the whole point of the region, and it is one screen: the seed picker acquires a source it did not have, and that source is the default.



LOADED — "Seed from where?" FROM PAYLOAD is the default



All 12 words, from the payload, typed by nobody



UNLOADED — the question is not asked at all

The third frame is the control that makes the first two mean something: with nothing loaded, the "from where?" step does not appear and the operator lands directly on Input Seed. Same firmware, same build, same program — the only difference is the region.

Load Payload, the program

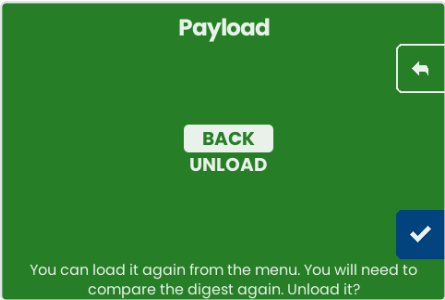
Position **8 of ten** in the top-level list, and the only one of the ten that the simulator could not walk at all until this run: SyswReader returned nil, so detection found nothing and every payload-fed screen above was unreachable in a browser.



Load Payload — the 8th of ten programs

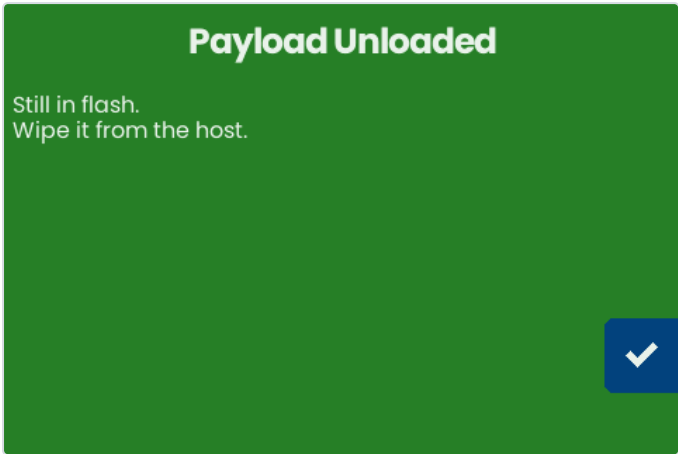


With one loaded: LOAD AGAIN / UNLOAD



UNLOAD asks, and says what it will cost: the digest again

On the machine — the way out



Payload Unloaded — "Still in flash. Wipe it from the host."

Eleven words that get the model right. UNLOAD is a RAM operation: the records are gone from this session, the region is exactly as it was, and the next boot will offer the payload again. The machine says so rather than letting "unload" be read as "erase".

The host half of that sentence is the `me sysw wipe` section earlier in this document. There is no device path to it, by design.

This screen shipped BLANK.

The title drew and the body did not, for a fortnight, because every GUI test asserted on text *submitted* to the op tree rather than pixels *drawn*. Filed as F-151; the raster assertion added with the fix was calibrated against the real defect — the broken body drew 2652 px, the fixed one 6688, and a floor of 2000 (the first guess) passed the defect.

One thing the machine does not tell you

The home screen is **byte-for-byte identical** whether a payload is loaded, was loaded and then unloaded, or was never offered:

```
$ sha256sum shots/p04-main-menu.png shots/p14-menu-after-unload.png shots/p18-menu-after-skip.png
f773d610f050ad9983c427635f25df6d80893a8c60ec2cef7efaedc2ab134204 p04-main-menu.png (loaded)
f773d610f050ad9983c427635f25df6d80893a8c60ec2cef7efaedc2ab134204 p14-menu-after-unload.png (unloaded)
f773d610f050ad9983c427635f25df6d80893a8c60ec2cef7efaedc2ab134204 p18-menu-after-skip.png (never loaded)
```

So an operator who walks away and comes back cannot tell whether a seed is currently in the machine's RAM without entering Load Payload and reading which two options it offers. Filed as **F-155**.

What this run turned up

- F-153 — me sysw pack's refusal named a cause that did not apply, and its record index is 0-based and unlabelled.**

Both hit on the first invocation of this journey. The message is FIXED (1538ef0): it now names the reserved prefix and the xxd remedy, states the index base, and still never prints the body. What remains filed is the base itself — --in filters blank lines, so the index is not a line number either, and making it one is a larger change than it looks.
- F-154 — the tenth program's dot is drawn underneath the firmware version line.**

Measured on the framebuffer, not eyeballed: the dot row runs to x≈322 and the "Firmware: emu" line starts at x≈305. It was fine at nine programs. Load Payload is the tenth, so the overlap arrived with the feature this document is about.
- F-155 — the home screen cannot tell you whether a payload is loaded.**

Three states, one identical frame, sha256 above. The machine knows; it just does not say. This matters more than a cosmetic issue because the thing being held is a seed.
- F-156 — the two published journeys cannot be regenerated by the commands their own README gives.**

transcript_pathological.sh and build_pdf_pathological.py both read inputs/, which now holds the other document's twelve cosigners; the pathological wallet's files moved to inputs-pathological/ and the scripts did not follow. build_pdf_pathological.py also reads its CSS from design/journey/build_pdf.py — a directory that does not exist — and opens a keys.json that was never committed. Neither script ran again after the rename, so nothing said so. This document's builder resolves every path it uses and produces the PDF itself.

Corrections to the two earlier journeys

Recorded here, and in design/journeys/README.md, because the published PDFs state them wrongly and a reader arrives at those first. Each is a follow-up with the measurement behind it.

#	what the earlier documents say or imply	what is true
F-131	"Recovery: any 3 of 11 signing keys + md1"	Not a 3-of-11. Four tiers, eight distinct minimal key-sets, each with its own timelock, two also needing a hash preimage. False in both directions.
F-132	the engraved set is a complete backup	Tiers 1–2 need the 32-byte preimage X to spend. Measured: it appears in 0 backup strings, and nothing in the checklist mentions it.
F-133	the tiers degrade weakest-last	INVERTED. Tier 4 (1-of-3) matures at 365.00 d; tier 3 (2-of-2) at ≈455 d. The weakest key-set unlocks ~90 days FIRST .
F-134	the wallet is 26 plates	26, 38 or 58 depending on an md1-form flag the operator is never shown. The default is the 58 one.
F-130	a restored wallet reproduces the descriptor	Keys are byte-identical and addresses unaffected, but restored xpubs lose depth/parent/child, so the descriptor string and its checksum differ.
F-136	md encode auto-chunks	It does not; it fails and tells you to retry with --force-chunked. Two places say otherwise, including the flag's own help.
F-127 F-128	the key-card stub comes from the md1	--from-md1 cannot read a chunked md1 at all (version 9, expected 4), and the stub mk embeds is the template-id , not the WalletPolicyId the spec names.
F-137	—	Encoder has no depth guard where the decoder does, so an encodable-but-undecodable card may be expressible. Unconfirmed ; carried on the codec lens's authority and flagged as such.
F-139	CORPUS.md §C6 is TBD	This wallet is the answer: 182 data symbols against a cap of 80.
F-140	compare-cost: taproot costs +127..+131 vB	True delta is +1..+6 vB . One side counts its script and the other does not, so the comparison points the wrong way.

- F-147, which is a habit rather than a document defect.**

cmd && echo OK prints nothing when cmd fails, and nothing reads as absence rather than failure. Its cousin bit this transcript while it was being written: nix ... | head -2 reported [exit 0] for a nix: command not found, because \$? was head's. Fixed by removing the pipe; the run () helper prints the real exit code unconditionally, and section 11 of the transcript says so where it happened.

Reproducing this document

```
cd design/journeys
mkdir -p out shots
bash transcript_payload.sh > out/transcript_payload.txt 2>&1 # every CLI block

# the device frames, in a browser (nothing here is a browser screenshot):
(cd ../../../../seedhammer/cmd/emu && ./build.sh) # inside the flake
(cd ../../../../seedhammer/cmd/emu && python3 -m http.server 8731 --bind 127.0.0.1) &
python3 shot_server.py "$PWD/shots" 8732 http://127.0.0.1:8731 &
# then drive http://127.0.0.1:8731/index.html and POST canvas.toDataURL()
# to :8732 for each frame -- see the frame names in build_pdf_payload.py

python3 build_pdf_payload.py # this PDF
```

out/ and shots/ are not committed; the PDF is the deliverable. transcript_payload.sh, build_pdf_payload.py and inputs-payload/ are.

Provenance of the test payload

The container in cmd/emu/sysw_test_payload.bin is the exact output of the me sysw pack in section 3, and section 6 of the transcript proves it with cmp and sha256sum. It is confined to the G00S=js build: TestSyswTestPayloadIsConfinedToJSOnlyFiles walks the module and fails if any file outside three named js-only ones mentions it, with a checked < 50 floor so a misrooted walk cannot pass vacuously. A shipped SeedHammer II must never boot carrying a payload somebody else packed.

Every screenshot is the emulator's own 480x320 framebuffer, POSTed as canvas.toDataURL() to a localhost receiver pinned to one origin — not a browser screenshot, so there is no chrome, no CSS scaling and no cropping guesswork. Every CLI block is real stdout+stderr with its true exit code; the only edit is shortening absolute paths to fit the column.